

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: Database Management Systems
COURSE CODE: CSA05

COURSE OUTCOME COVERED

CO2: Apply the relational model, relational algebra, SQL query structures, and views to solve database querying and data manipulation problems. (BL3, BL6)

Activity Tool : Analytical Problem Solving (High)
Assessment Tool Weightage: 35%

Dr.G.Ayyappan

QUESTIONS			Total Marks: 50
Q.No	Question	Bloom's Level	Marks
1	Apply relational algebra operations (σ , π , $\cup$, $-$, $\times$, $\bowtie$) on the given Employee and Department relations to retrieve required records.	BL3 – Apply	5
2	Write SQL DDL statements to create STUDENT(SID, Name, DOB, DeptID) and DEPARTMENT(DeptID, DeptName) tables with all appropriate constraints.	BL3 – Apply	5
3	Formulate SQL queries using GROUP BY, HAVING, and aggregate functions (COUNT, SUM, AVG, MAX, MIN) on a given Sales dataset.	BL3 – Apply	5
4	Write a correlated sub-query to find employees earning more than the average salary of their own department.	BL3 – Apply	5
5	Apply all types of JOIN operations (INNER, LEFT OUTER, RIGHT OUTER, FULL OUTER, CROSS) on Employee and Project tables.	BL3 – Apply	5
6	Create a view 'DeptSalaryView' that displays department name, total employees, and average salary. Write a query on this view.	BL6 – Create	5
7	Construct a relational algebra expression for the following: Find names of students who are enrolled in all courses offered.	BL6 – Create	5

8	Write SQL DML statements (INSERT, UPDATE, DELETE) with integrity constraint enforcement for a given scenario.	BL3 – Apply	5
9	Apply tuple relational calculus to express a query: Find all employees who work in the 'IT' department and earn more than 50000.	BL3 – Apply	5
10	Design a relational schema for an Airline Reservation System and write 5 SQL queries demonstrating joins, sub-queries, and views.	BL6 – Create	5

Code of Conduct:

*I **C. Sravanthi/192525463** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student: C. Sravanthi

RUBRICS FOR CALCULATION:

Numerical Problems					
Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	20%	Clearly interprets problem, identifies all parameters and requirements accurately	Understands problem with minor gaps	Partial understanding; misses key elements	Misinterprets or unable to understand problem
Method Selection	20%	Selects most appropriate method/formula with strong justification	Selects correct method with minor errors	Inappropriate or partially correct method	Unable to select method

Solution Process	25%	Logical, systematic steps with correct approach throughout	Mostly correct steps with minor mistakes	Disorganized steps; multiple errors	No clear process
Accuracy of Calculation	20%	All calculations accurate; correct final answer	Minor calculation errors	Frequent errors; incorrect final answer	No valid calculations
Interpretation of Results	15%	Clearly interprets results with units and engineering relevance	Basic interpretation with minor omissions	Limited or unclear interpretation	No interpretation

ANSWERS:

Question 1-

Apply relational algebra operations (σ , π , $\cup$, $-$, $\times$, $\bowtie$) on the given Employee and Department relations to retrieve required records.

Answer:

Assume the following relations:

EMPLOYEE(EmpID, EmpName, Salary, DeptID)

EmpID	EmpName	Salary	DeptID
101	Amit	60000	D1
102	Riya	45000	D2
103	Rahul	55000	D1
104	Sneha	40000	D3

DEPARTMENT(DeptID, DeptName)

DeptID	DeptName
D1	IT
D2	HR
D3	Finance

(a) Selection (σ)

Find employees with salary greater than ₹50,000.

$\sigma_{\text{Salary} > 50000}(\text{EMPLOYEE})$

(b) Projection (π)

Display only employee names.

$\pi_{\text{EmpName}}(\text{EMPLOYEE})$

(c) Union ($\cup$)

Suppose

$\text{IT_EMP} = \{\text{Amit, Rahul}\}$

$\text{HR_EMP} = \{\text{Riya}\}$

$\text{IT_EMP} \cup \text{HR_EMP}$

Result:

$\{\text{Amit, Rahul, Riya}\}$

(d) Difference ($-$)

$\text{IT_EMP} - \text{HR_EMP}$

Result:

$\{\text{Amit, Rahul}\}$

(e) Cartesian Product ($\times$)

$\text{EMPLOYEE} \times \text{DEPARTMENT}$

Result:

$4 \times 3 = 12$ tuples

(f) Join ($\bowtie$)

$\text{EMPLOYEE} \bowtie \text{EMPLOYEE.DeptID} = \text{DEPARTMENT.DeptID} \text{DEPARTMENT}$

Result:

EmpName	DeptName
Amit	IT
Rahul	IT
Riya	HR
Sneha	Finance

Question 2

Write SQL DDL statements to create STUDENT(SID, Name, DOB, DeptID) and DEPARTMENT(DeptID, DeptName) tables with all appropriate constraints.

Answer:

```
</> SQL

CREATE TABLE DEPARTMENT (
    DeptID INT PRIMARY KEY,
    DeptName VARCHAR(50) NOT NULL UNIQUE
);

CREATE TABLE STUDENT (
    SID INT PRIMARY KEY,
    Name VARCHAR(50) NOT NULL,
    DOB DATE NOT NULL,
    DeptID INT,
    FOREIGN KEY (DeptID)
    REFERENCES DEPARTMENT(DeptID)
);
```

Question 3

Formulate SQL queries using GROUP BY, HAVING, and aggregate functions (COUNT, SUM, AVG, MAX, MIN) on a given Sales dataset.

Answer:

Assume the **SALES** table contains:

Sale ID	Product	Amount	Region
1	Laptop	50000	South
2	Laptop	60000	North
3	Mouse	1000	South
4	Keyboard	2000	North
5	Laptop	55000	South

1. COUNT

```
SELECT Product, COUNT(*) AS TotalSales
FROM SALES
GROUP BY Product;
```

2. SUM

```
SELECT Product, SUM(Amount) AS TotalAmount
FROM SALES
GROUP BY Product;
```

3. AVG

```
SELECT Product, AVG(Amount) AS AverageAmount
FROM SALES
GROUP BY Product;
```

4. MAX

```
SELECT Product, MAX(Amount) AS MaximumSale
FROM SALES
GROUP BY Product;
```

5. MIN

```
SELECT Product, MIN(Amount) AS MinimumSale
FROM SALES
GROUP BY Product;
```

6. HAVING Clause

```
SELECT Product, SUM(Amount) AS TotalAmount
```

FROM SALES
GROUP BY Product
HAVING SUM(Amount) > 50000;

Question 4

Write a correlated sub-query to find employees earning more than the average salary of their own department.

Answer:

Assume the **EMPLOYEE** table:

EmpID	EmpName	Salary	DeptID
101	Amit	60000	D1
102	Riya	45000	D2
103	Rahul	55000	D1
104	Sneha	40000	D3

SQL

```
SELECT EmpID, EmpName, Salary, DeptID
FROM EMPLOYEE E1
WHERE Salary >
(
SELECT AVG(Salary)
FROM EMPLOYEE E2
WHERE E1.DeptID = E2.DeptID
);
```

Question 5

Apply all types of JOIN operations (INNER, LEFT OUTER, RIGHT OUTER, FULL OUTER, CROSS) on Employee and Project tables.

Question 4

Write a correlated sub-query to find employees earning more than the average salary of their own department.

Answer:

Assume the **EMPLOYEE** table:

EmpID	EmpName	Salary	DeptID
101	Amit	60000	D1
102	Riya	45000	D2
103	Rahul	55000	D1
104	Sneha	40000	D3

```
SELECT EmpID, EmpName, Salary, DeptID
FROM EMPLOYEE E1
WHERE Salary >
(
SELECT AVG(Salary)
FROM EMPLOYEE E2
WHERE E1.DeptID = E2.DeptID
);
```

Question 5

Apply all types of JOIN operations (INNER, LEFT OUTER, RIGHT OUTER, FULL OUTER, CROSS) on Employee and Project tables.

Answer:

Assume the tables:

EMPLOYEE

EmpID	EmpName	ProjectID
101	Amit	P1
102	Riya	P2
103	Rahul	P3
104	Sneha	NULL

PROJECT

ProjectID	ProjectName
P1	Payroll
P2	Website

ProjectID	ProjectName
P3	Inventory
P4	Library

1. INNER JOIN

```
SELECT E.EmpName, P.ProjectName
FROM EMPLOYEE E
INNER JOIN PROJECT P
ON E.ProjectID = P.ProjectID;
```

2. LEFT OUTER JOIN

```
SELECT E.EmpName, P.ProjectName
FROM EMPLOYEE E
LEFT JOIN PROJECT P
ON E.ProjectID = P.ProjectID;
```

3. RIGHT OUTER JOIN

```
SELECT E.EmpName, P.ProjectName
FROM EMPLOYEE E
RIGHT JOIN PROJECT P
ON E.ProjectID = P.ProjectID;
```

4. FULL OUTER JOIN

```
SELECT E.EmpName, P.ProjectName
FROM EMPLOYEE E
FULL OUTER JOIN PROJECT P
ON E.ProjectID = P.ProjectID;
```

5. CROSS JOIN

```
SELECT E.EmpName, P.ProjectName
FROM EMPLOYEE E
CROSS JOIN PROJECT P;
```

Question 6

Create a view 'DeptSalaryView' that displays department name, total employees, and average salary. Write a query on this view.

Answer:

```
CREATE VIEW DeptSalaryView AS
SELECT D.DeptName,
COUNT(E.EmpID) AS TotalEmployees,
```

```
AVG(E.Salary) AS AverageSalary
FROM EMPLOYEE E
JOIN DEPARTMENT D
ON E.DeptID = D.DeptID
GROUP BY D.DeptName;
```

Query on the View

```
SELECT *
FROM DeptSalaryView;
```

Question 7

Construct a relational algebra expression to find names of students who are enrolled in all courses offered.

Answer:

Assume the relations:

- STUDENT(SID, Name)
- COURSE(CID, CourseName)
- ENROLLMENT(SID, CID)

Relational Algebra:

$$\pi_{\text{Name}}(\text{STUDENT} \bowtie (\pi_{\text{SID}, \text{CID}}(\text{ENROLLMENT}) \div \pi_{\text{CID}}(\text{COURSE})))$$

This expression uses the **division** ($\div$) operator to find students enrolled in **every** course.

Question 8

Write SQL DML statements (INSERT, UPDATE, DELETE) with integrity constraint enforcement for a given scenario.

Answer:

```
INSERT
INSERT INTO STUDENT
(SID, Name, DOB, DeptID)
VALUES
(101, 'Rahul', '2005-04-10', 1);
```

```
UPDATE
```

```
UPDATE STUDENT
SET DeptID = 2
WHERE SID = 101;
```

DELETE

```
DELETE FROM STUDENT
WHERE SID = 101;
```

These statements assume that the foreign key (DeptID) exists in the DEPARTMENT table, ensuring referential integrity.

Question 9

Apply tuple relational calculus to express a query: Find all employees who work in the 'IT' department and earn more than 50000.

Answer:

$$\{t \mid \text{EMPLOYEE}(t) \wedge t.\text{DeptName} = \text{'IT'} \wedge t.\text{Salary} > 50000\}$$

This tuple relational calculus expression returns all employee tuples where the department is **IT** and the salary is greater than **50000**.

Question 10

Design a relational schema for an Airline Reservation System and write 5 SQL queries demonstrating joins, sub-queries, and views.

Answer

Relational Schema

PASSENGER

Attribute	Data Type	Constraint
PassengerID	INT	PRIMARY KEY
PassengerName	VARCHAR(50)	NOT NULL
Phone	VARCHAR(15)	NOT NULL

FLIGHT

Attribute	Data Type	Constraint
FlightID	INT	PRIMARY KEY

Attribute	Data Type	Constraint
FlightName	VARCHAR(50)	NOT NULL
Source	VARCHAR(50)	NOT NULL
Destination	VARCHAR(50)	NOT NULL

BOOKING

Attribute	Data Type	Constraint
BookingID	INT	PRIMARY KEY
PassengerID	INT	FOREIGN KEY REFERENCES PASSENGER(PassengerID)
FlightID	INT	FOREIGN KEY REFERENCES FLIGHT(FlightID)
JourneyDate	DATE	NOT NULL

SQL Query 1 – INNER JOIN

```
SELECT P.PassengerName, F.FlightName
FROM PASSENGER P
JOIN BOOKING B
ON P.PassengerID = B.PassengerID
JOIN FLIGHT F
ON B.FlightID = F.FlightID;
```

SQL Query 2 – JOIN with WHERE Clause

```
SELECT P.PassengerName, F.Destination
FROM PASSENGER P
JOIN BOOKING B
ON P.PassengerID = B.PassengerID
JOIN FLIGHT F
ON B.FlightID = F.FlightID
WHERE F.Destination = 'Chennai';
```

SQL Query 3 – Subquery

```
SELECT PassengerName
FROM PASSENGER
WHERE PassengerID IN
(
    SELECT PassengerID
```

```
FROM BOOKING  
);
```

SQL Query 4 – Create View

```
CREATE VIEW FlightDetails AS  
SELECT P.PassengerName,  
       F.FlightName,  
       F.Source,  
       F.Destination  
FROM PASSENGER P  
JOIN BOOKING B  
ON P.PassengerID = B.PassengerID  
JOIN FLIGHT F  
ON B.FlightID = F.FlightID;
```

SQL Query 5 – Query the View

```
SELECT *  
FROM FlightDetails;
```